



**МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ**  
**имени М.В.Ломоносова**



---

**Факультет вычислительной математики и кибернетики**

**Компьютерный практикум по учебному курсу**  
**«ВВЕДЕНИЕ В ЧИСЛЕННЫЕ МЕТОДЫ»**  
**ЗАДАНИЕ № 1**

**ОТЧЕТ**

**о выполненном задании**

студентки 219 учебной группы факультета ВМК МГУ

Учар Айгуль Хулусиевны

гор. Москва

2023 г.

# Решение систем линейных алгебраических уравнений методом Гаусса и методом Гаусса с выбором главного элемента

## Цель работы

Изучить классический метод Гаусса (также метод Гаусса с выбором главного элемента), применяемый для решения системы линейных алгебраических уравнений.

## Постановка задачи

Дана система уравнений  $Ax = f$  порядка  $n \times n$  с невырожденной матрицей  $A$ . Требуется написать программу, решающую систему линейных алгебраических уравнений заданного пользователем размера ( $n$  – параметр программы) методом Гаусса и методом Гаусса с выбором главного элемента.

Необходимо предусмотреть возможность задания элементов матрицы системы и ее правой части как во входном файле данных, так и путем задания специальных формул.

## Цели и задачи практической работы

- 1) Решить заданную СЛАУ методом Гаусса и методом Гаусса с выбором главного элемента;
- 2) Вычислить определитель матрицы  $\det(A)$ ;
- 3) Вычислить обратную матрицу  $A^{-1}$ ;
- 4) Определить число обусловленности  $M_A = \|A\| \times \|A^{-1}\|$ ;
- 5) Исследовать вопрос вычислительной устойчивости метода Гаусса;
- 6) Подтвердить правильность решения СЛАУ системой тестов.

# Описание метода решения

## Метод Гаусса

Решение системы линейных алгебраических уравнений (СЛАУ) методом Гаусса включает в себя два основных этапа. На первом этапе выполняется прямой ход, где с использованием элементарных преобразований строк система приводится к треугольной форме. На втором этапе реализуется обратный ход, заключающийся в последовательном вычислении компонентов решения СЛАУ.

Прямой ход:

- Рассмотрим начальный элемент в первой строке. В случае, если этот элемент равен нулю, производим обмен данной строки с той строкой, где первый элемент отличен от нуля. Поскольку предполагается, что матрица является невырожденной, такая строка обязательно будет найдена. Затем все элементы в первой строке делятся на начальный элемент, и коэффициенты переопределяются:

$$c_{12} = \frac{a_{12}}{a_{11}}, \dots, c_{1n} = \frac{a_{1n}}{a_{11}}, y_1 = \frac{f_1}{a_{11}}$$

- Теперь вычтем из каждой строки, начиная со второй, первую строку, умноженную на  $a_{i1}$  ( $i = 2, \dots, n$ ), таким образом обнулим первые элементы в полученных строках, после чего вновь переобозначим коэффициенты

$$a_{ij}^1 = a_{ij} - \frac{a_{i1}a_{1j}}{a_{11}}, f_i^1 = f_i - \frac{a_{i1}f_1}{a_{11}}, i, j = 2 \dots n$$

- Повторим вышеперечисленные операции для  $n - 1$  строк (уравнений), начиная со второй;
- Проделав такую операцию  $n - 1$  раз, получим верхнюю треугольную матрицу, соответствующую следующей СЛАУ:

$$\begin{cases} x_1 + c_{12}x_2 + c_{13}x_3 + \dots + c_{1n}x_n = y_1 \\ x_2 + c_{23}x_3 + \dots + c_{2n}x_n = y_2 \\ \dots \\ x_{n-1} + c_{n-1,n}x_n = y_{n-1} \\ x_n = y_n \end{cases}$$

Обратный ход:

- Начиная с последнего уравнения, последовательно находим неизвестные из полученной системы:

$$\begin{cases} x_n = y_n \\ x_{n-1} = -(c_{n-1,n}x_n) + y_{n-1} \\ \dots \\ x_1 = -(c_{12}x_2 + \dots + c_{1n}x_n) + y_1 \end{cases}$$

## Метод Гаусса с выбором главного элемента

Этот метод аналогичен стандартному методу Гаусса, за исключением того, что на каждом шаге  $i$ -ого этапа (где  $i$  - номер текущего шага) выбирается максимальный по модулю элемент в  $i$ -ом столбце. Если такой элемент находится в  $j$ -ой строке, то производится обмен местами  $i$ -ой и  $j$ -ой строк.

## Вычисление определителя

После получения верхнетреугольной матрицы определитель вычисляется как произведение диагональных элементов. Определитель полученной треугольной матрицы равен 1, поскольку преобразования заключаются в деление строк на соответствующие коэффициенты на каждой итерации. Также стоит отметить, что перестановка строк меняет знак определителя. Таким образом, определитель исходной матрицы вычисляется следующим образом:

$$\det(A) = (-1)^{cnt} a_{11} a_{22}^1 \dots a_{nn}^{n-1}$$

## Нахождение обратной матрицы

Обратная матрица находится с использованием метода Гаусса-Жордана. Записываем рядом матрицу  $A$  и единичную матрицу  $I$ :  $(A|I)$ . С помощью элементарных преобразований строк приводим к виду:  $(I|A^{-1})$ , где  $A^{-1}$  - искомая обратная матрица.

## Вопрос вычислительной устойчивости метода Гаусса

Положим  $M_A = \|A\| \|A^{-1}\|$  - число обусловленности матрицы  $A$ , где  $\|A\|$  - норма матрицы (используется норма  $\|A\|_\infty = \max_i \sum_j |a_{ij}|$ ), тогда допустим, правая часть исходной системы подверглась возмущению  $\delta f$  и стала равной  $\tilde{f} = f + \delta f$ , откуда имеем решение новой системы:  $A\tilde{x} = \tilde{f}, \tilde{x} = x + \delta x$ .

Для  $\|\delta x\|, \|\delta f\|$  верно:  $\frac{\|\delta x\|}{\|x\|} \leq M_A \frac{\|\delta f\|}{\|f\|}$ , тем самым число обусловленности

позволяет провести оценку относительной погрешности решения через относительную погрешность возмущения правой части. Матрицы с высоким числом обусловленности и соответствующие системы линейных алгебраических уравнений (СЛАУ) называются плохо обусловленными. Это означает, что при решении таких систем могут возникнуть значительные погрешности в вычислениях, что делает их более чувствительными к небольшим изменениям входных данных или коэффициентов матрицы.

## Описание программы

Исходный код программы сдается совместно с отчетом в файле ger1-1.ру.

- *def create\_arr(n)*: функция по заданному числу  $n$  формирует и возвращает матрицу, соответствующую СЛАУ, по следующей формуле:

$$A_{ij} = \begin{cases} q_M^{i+j} + 0.1 \cdot (j-i), & i \neq j, \\ (q_M - 1)^{i+j}, & i = j, \end{cases}$$

где  $q_M = 1.001 - 2 \cdot M \cdot 10^{-3}$ ,  $i, j = 1, \dots, n$ .

$$f_i = x \cdot \exp\left(\frac{x}{i}\right) \cdot \cos\left(\frac{x}{i}\right)$$

- *def print\_right\_side(m)*: данной функции на вход подается матрица, соответствующая СЛАУ, функция последовательно выводит элементы правой части СЛАУ;
- *def det(m)*: функции на вход подается матрица, функция же методом Гаусса находит ее определитель и возвращает значение;
- *def gauss(m)*: функция методом Гаусса приводит заданную матрицу к диагональной форме и возвращает полученную матрицу;
- *def pivot\_gauss(m)*: функция методом Гаусса с выбором главного элемента приводит заданную матрицу к диагональной форме и возвращает полученную матрицу;
- *def inverse\_arr(m)*: функция находит методом Гаусса-Жордана обратную матрицу к заданной и возвращает полученную матрицу;
- *def norm(m)*: функция находит и возвращает бесконечную норму полученной матрицы;
- *def condition(m)*: функция находит и возвращает число обусловленности заданной матрицы

## Исходный код

```
import numpy as np

EPS = 1e-12

def create_arr(x): # заполнение матрицы по заданной формуле
    arr = []
    M = 6
    n = 100
    q = 1.001 - 2 * M * 10**(-3);
    for i in range(1, n+1):
        arr.append([])

        for j in range(1, n+1):
            if i != j:
                arr[i-1].append(q ** (i+j) + 0.1*(j-i))
            else :
                arr[i-1].append((q-1)**(i+j))

        arr[i-1].append(x * np.exp(x/i) * np.cos(x/i))

    return arr

def print_right_side(m): # вывод правой части СЛАУ
    n = len(m)

    for i in range(n):
        print('x', end = '')
        print(i + 1, '=', m[i][n])

    print()

def det(m): # вычисление определителя матрицы
    new_m = np.copy(m)
    res = 1
    n = len(m)
    for i in range(n - 1):
        if len(np.nonzero(abs(new_m[i:, i]) > EPS)[0]) == 0:
            return 0
        nonzero = np.nonzero(abs(new_m[i:, i]) > EPS)[0][0] + i
        new_m[[i, nonzero]] = new_m[[nonzero, i]]
        if (i != nonzero):
            res = res * -1
        res = res * new_m[i][i]
```

```

        new_m[i] = new_m[i] / new_m[i][i]
        new_m[i + 1:, :] = new_m[i + 1:, :] - np.matmul(np.transpose(new_m[i + 1:,
                                                                           i])[ :, np.newaxis], new_m[i, np.newaxis])
    res = res * new_m[n - 1][n - 1]
    return res

```

```

def gauss(m): # нахождение решения системы методом Гаусса

```

```

    new_m = np.copy(m)
    n = len(m)

```

```

    for i in range(n - 1):
        nonzero = np.nonzero(abs(new_m[i:, i]) > EPS)[0][0] + i

        new_m[[i, nonzero]] = new_m[[nonzero, i]]
        new_m[i] = new_m[i] / new_m[i][i]
        new_m[i + 1:, :] = new_m[i + 1:, :]
        - np.matmul(np.transpose(new_m[i + 1:, i])[ :, np.newaxis],
                    new_m[i, np.newaxis])

```

```

    for i in range(n - 1, -1, -1):
        new_m[i] = new_m[i] / new_m[i][i]
        new_m[:i, :] = new_m[:i, :] - np.matmul(np.transpose(new_m[:i, i])[ :,
                                                                , np.newaxis], new_m[i, np.newaxis])

```

```

    return new_m

```

```

def pivot_gauss(m): # решение системы методом Гаусса с выбором главного элемента
    new_m = np.copy(m)
    n = new_m.shape[0]

    for i in range(n - 1):
        max = np.argmax(abs(new_m[i:, i]) > EPS) + i

        new_m[[i, max]] = new_m[[max, i]]

        new_m[i] = new_m[i] / new_m[i][i]
        new_m[i + 1:, :] = new_m[i + 1:, :]
        - np.matmul(np.transpose(new_m[i + 1:, i])[:, np.newaxis], new_m[i, np.newaxis])

    for i in range(n - 1, -1, -1):
        new_m[i] = new_m[i] / new_m[i][i]
        new_m[:, i, :] = new_m[:, i, :]
        - np.matmul(np.transpose(new_m[:, i, i])[:, np.newaxis], new_m[i, np.newaxis])

    return new_m

def inverse_array(m): # нахождение обратной матрицы методом Гаусса-Жордана
    n = len(m)
    return gauss(np.concatenate((m[:, :-1], np.eye(n, n)), axis = 1))[:, n:]

def norm(m): # нахождение нормы матрицы
    return np.max(np.sum(abs(m[:, : len(m)]), axis = 1))

def condition(m): # нахождение числа обусловленности матрицы
    return norm(m) * norm(inverse_array(m))

def main():
    print('How you want to enter the matrix?')
    print('>>Manually (1)')
    print('>>Using the formula (2)')

    mode = int(input('Choose mode 1 or 2 = '))

```

```

if mode == 1:
    n = int(input('n = '))
    m = []
    for i in range(n):
        m.append([float(j) for j in input('current row: ').split()])

else:
    n = 100
    x = int(input('x = '))
    m = create_arr(x)

m = np.array(m, dtype = float)
det_m = det(m)

print("\nSolution using the Gauss method")

if det_m != 0:
    print_right_side(gauss(m))
else:
    print('Determinant is equal to 0\n')

print("Solution using the pivot Gauss method")

if det_m != 0:
    print_right_side(pivot_gauss(m))
else:
    print('Determinant is equal to 0\n')

print("Determinant of a matrix:")
print(det_m, end = '\n\n')

print("Inverse matrix:")

if det_m != 0:
    print(inverse_array(m), end = '\n\n')
else:
    print('Determinant is equal to 0\n')

print("Condition number:")

if det_m != 0:
    print(condition(m))
else:
    print('Determinant is equal to 0\n')

if __name__ == '__main__':
    main()

```

# Тесты

## Приложение 1 вариант 7

### Тест 1

$$\begin{cases} 2x_1 - x_2 - 6x_3 + 3x_4 = -1, \\ 7x_1 - 4x_2 + 2x_3 - 15x_4 = -32, \\ x_1 - 2x_2 - 4x_3 + 9x_4 = 5, \\ x_1 - x_2 + 2x_3 - 6x_4 = -8. \end{cases}$$

|                       | Результат программы                                                                                                     | Реальное значение                                                                                                       |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Решение СЛАУ          | (-3.0, 1.0, 7.0, 1.3333...)                                                                                             | (-3.0, 1.0, 7.0, 1.3333...)                                                                                             |
| Определитель матрицы  | -251.99999999999998                                                                                                     | -252                                                                                                                    |
| Обратная матрица      | -0.214... 0.381... -0.071... -1.167...<br>-0. 0.111... -0.333... -0.778...<br>-0. -0. -1. -1.5<br>-0. -0. -0. -0.167... | -0.214... 0.381... -0.071... -1.167...<br>-0. 0.111... -0.333... -0.778...<br>-0. -0. -1. -1.5<br>-0. -0. -0. -0.167... |
| Число обусловленности | 70.0                                                                                                                    | 70                                                                                                                      |

### Тест 2

$$\begin{cases} x_1 - 2x_2 + x_3 + x_4 = 0, \\ 2x_1 + x_2 - x_3 - x_4 = 0, \\ 3x_1 - x_2 - 2x_3 + x_4 = 0, \\ 5x_1 + 2x_2 - x_3 + 9x_4 = -10. \end{cases}$$

|                       | Результат программы                                                                                                | Реальное значение                                                                                          |
|-----------------------|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Решение СЛАУ          | (-0.254..., -9090., -0.588..., -1.11..)                                                                            | (-0.254..., -9090., -0.588..., -1.11..)                                                                    |
| Определитель матрицы  | -117.99999999999993                                                                                                | -118                                                                                                       |
| Обратная матрица      | 0.2797.. 0.424.. -0.085.. 0.025..<br>0. 0.515.. -0.3030.. 0.0909..<br>-0. -0. -0.529.. 0.059..<br>0. 0. 0. 0.111.. | 0.2797.. 0.424.. -0.085.. 0.025..<br>0 0.515.. -0.3030.. 0.0909..<br>0 0 -0.529.. 0.059..<br>0 0 0 0.111.. |
| Число обусловленности | 15.4545...                                                                                                         | 15.4545..                                                                                                  |

### Тест 3

$$\begin{cases} 8x_1 + 6x_2 + 5x_3 + 2x_4 = 21, \\ 3x_1 + 3x_2 + 2x_3 + x_4 = 10, \\ 4x_1 + 2x_2 + 3x_3 + x_4 = 8, \\ 7x_1 + 4x_2 + 5x_3 + 2x_4 = 18. \end{cases}$$

|                       | Результат программы                                                                                                                                                | Реальное значение                                                                                                                                                  |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Решение СЛАУ          | (3.00..., 6.66e-16, -2.0, 9.0)                                                                                                                                     | (3, 0, -5, 11)                                                                                                                                                     |
| Определитель матрицы  | -242.0                                                                                                                                                             | -242.0                                                                                                                                                             |
| Обратная матрица      | 0.65 ... -0.16 ... -0.10 ... -0.01 ...<br>-0.21 ... 0.37 ... -0.00 ... 0.03 ...<br>-0.08 ... -0.04 ... 0.22 ... -0.00 ...<br>0.09 ... 0.09 ... -0.09 ... -0.09 ... | 0.65 ... -0.16 ... -0.10 ... -0.01 ...<br>-0.21 ... 0.37 ... -0.00 ... 0.03 ...<br>-0.08 ... -0.04 ... 0.22 ... -0.00 ...<br>0.09 ... 0.09 ... -0.09 ... -0.09 ... |
| Число обусловленности | 16.95...                                                                                                                                                           | 16.95...                                                                                                                                                           |

Приложение 2 пример 2 вариант 6

#### Тест 4

Элементы матрицы А вычисляются по формулам:

$$A_{ij} = \begin{cases} q_M^{i+j} + 0.1 \cdot (j-i), & i \neq j, \\ (q_M - 1)^{i+j}, & i = j, \end{cases}$$

где  $q_M = 1.001 - 2 \cdot M \cdot 10^{-3}$ ,  $i, j = 1, \dots, n$ .

$$f_i = x \cdot \exp\left(\frac{x}{i}\right) \cdot \cos\left(\frac{x}{i}\right)$$

$M = 6, N = 100$

Результаты программы:

Определитель - 2.2391916183682835e-43

Полученные результаты близки к результатам работы библиотечных функций, стоит учитывать, что с увеличением размерности матрицы растет и ее число обусловленности, что сказывается на увеличении погрешности при вычислениях.

## Вывод

Мы рассмотрели метод Гаусса и метод Гаусса с выбором главного элемента для решения систем линейных алгебраических уравнений (СЛАУ). Важно отметить, что метод Гаусса при увеличении размерности матрицы может проявить неустойчивость, и хотя его модифицированный вариант позволяет улучшить точность решения, метод Гаусса с выбором главного элемента также при возрастающей размерности может привести к увеличению погрешности в вычислениях.

# Итерационные методы решения систем линейных алгебраических уравнений на примере методов Зейделя и верхней релаксации

## Цель работы

Исследование классических итерационных методов, таких как метод Зейделя и метод верхней релаксации, применяемых для численного решения систем линейных алгебраических уравнений (СЛАУ), является важным этапом в области вычислительной линейной алгебры. Особое внимание уделяется анализу скорости сходимости этих методов в зависимости от выбора итерационного параметра.

## Постановка задачи

Дана система уравнений  $Ax=f$  порядка  $n \times n$  с невырожденной матрицей  $A$ . Написать программу численного решения данной системы линейных алгебраических уравнений ( $n$  – параметр программы), использующую численный алгоритм итерационного метода верхней релаксации:

$(D + \omega A^{(-)}) \frac{x^{k+1} - x^k}{\omega} + Ax^k = f$ , где  $D, A^{(-)}$  – соответственно диагональная и нижняя треугольные матрицы,  $k$  – номер текущей итерации,  $\omega$  – итерационный параметр.

Необходимо предусмотреть возможность задания элементов матрицы системы и ее правой части как во входном файле данных, так и путем задания специальных формул.

## Цели и задачи практической работы

- 1) Решить заданную СЛАУ итерационным методом верхней релаксации;
- 2) Разработать критерий остановки итерационного процесса, гарантирующий получение приближенного решения исходной системы СЛАУ с заданной точностью;
- 3) Изучить скорость сходимости итераций к точному решению задачи, провести эксперименты с различными значениями итерационного параметра;
- 4) Правильность решения СЛАУ подтвердить системой тестов

## Описание метода решения

### Описание итерационного процесса

Итерационный процесс метода верхней релаксации имеет вид:

$$(D + \omega A^{(-)}) \frac{x^{k+1} - x^k}{\omega} + Ax^k = f$$

Где  $D = \begin{cases} a_{ij}, i = j \\ 0, i \neq j \end{cases}$  - диагональная матрица

$A^{(-)} = \begin{cases} a_{ij}, i > j \\ 0, i \leq j \end{cases}$  - нижняя треугольная матрица,

k - номер текущей итерации,  $\omega$  – итерационный параметр ( $0 < \omega < 2$ ).

Данная формула может быть записана в виде:

$$\left(\frac{1}{\omega}D + A^{(-)}\right)x^{k+1} + \left[\left(1 - \frac{1}{\omega}\right)D + H\right]x^k = f$$

$H = \begin{cases} a_{ij}, i < j \\ 0, i \geq j \end{cases}$  – верхняя треугольная матрица.

Откуда получаем формулу для вычисления  $x^{k+1}$ :

$$x_i^{k+1} + \omega \sum_{j=1}^{i-1} \frac{a_{ij}}{a_{ii}} x_j^{k+1} = (1 - \omega)x_i^k - \omega \sum_{j=i+1}^n \frac{a_{ij}}{a_{ii}} x_j^k + \omega \frac{f_i}{a_{ii}}$$

Значит, алгоритмически это выражение можно записать так:

$$x_i := x_i - \omega \sum_{j=1}^n \frac{a_{ij}}{a_{ii}} x_j + \omega \frac{f_i}{a_{ii}}$$

### Условие сходимости

Допустим, данная матрица является самосопряженной и положительно определенной, а также итерационный параметр ограничен следующим образом:  $0 < \omega < 2$ . Тогда по теореме Самарского итерационный процесс сходится к точному решению при любом выборе начального приближения  $x_0$ .

Если же вышеописанные условия не выполняются, итерационный процесс может не сходиться, в этом случае для невырожденной матрицы необходимо домножить слева СЛАУ на  $A^T$ , тогда полученная матрица окажется самосопряженной и положительно определенной, а решение полученной системы будет совпадать с решением исходной системы.

### **Критерий остановки итерационного процесса**

Для оценки погрешности полученного решения на очередной итерации используется невязка:  $\psi_k = Ax_k - f$ .

$\|\psi_k\| \sim \|x_k - x\|$ , следовательно, в качестве критерия остановки итерационного процесса можно взять норму невязки:  $\|\psi_k\| < \varepsilon$ , где  $\varepsilon$  — заданный параметр, характеризующий точность решения.

## Описание программы

Исходный код программы сдается совместно с отчетом в файле her1-2.ру.

- *def is\_pos(m)*: функция определяет, является ли данная матрица положительно определенной;
- *def is\_herm(m)*: функция определяет, является ли данная матрица самосопряженной;
- *def form\_matr(n)*: функция по заданному числу  $n$  формирует и возвращает матрицу, соответствующую СЛАУ, по следующей формуле:

$$A_{ij} = \begin{cases} q_M^{i+j} + 0.1 \cdot (j-i), & i \neq j, \\ (q_M - 1)^{i+j}, & i = j, \end{cases}$$

где  $q_M = 1.001 - 2 \cdot M \cdot 10^{-3}$ ,  $i, j = 1, \dots, n$ .

$$f_i = x \cdot \exp\left(\frac{x}{i}\right) \cdot \cos\left(\frac{x}{i}\right)$$

- *def residual(m, x)*: функция вычисляет и возвращает невязку на очередном шаге итерационного процесса:  $\psi_k = Ax_k - f$ ;
- *def relax(m, e, w)*: функция находит и возвращает решение СЛАУ итерационным методом верхней релаксации с заданным итерационным параметром  $w$  и заданной погрешностью  $e$ , а также выводит число итераций, которые потребовались для получения такого результата;
- *def det(m)*: функции на вход подается матрица, функция же методом Гаусса находит ее определитель и возвращает значение;
- *def norm(m)*: функция находит и возвращает бесконечную норму полученной матрицы

## Исходный код

---

```
import numpy as np
import math

EPS = 1e-12

def is_pos(m): # checking a matrix if it is positive-definite
    return np.all(np.linalg.eigvals(m[:, : -1]) > 0)

def is_herm(m): # checking a matrix if it is hermitian
    m_trans = m[:, : -1].transpose()
    return m[:, : -1].all() == m_trans.all()

def form_matr(x): # filling the matrix according to the given formula
    matr = []
    M = 6
    n = 100
    q = 1.001 - 2 * M * 10**(-3);
    for i in range(1, n+1):
        matr.append([])

        for j in range(1, n+1):
            if i != j:
                matr[i-1].append(q ** (i+j) + 0.1*(j-i))
            else :
                matr[i-1].append((q-1)**(i+j))

        matr[i-1].append(x * np.exp(x/i) * np.cos(x/i))

    return matr

def residual(m, x): # calculation of the residual of current result
    return m[:, : -1].dot(np.transpose([x])) - m[:, -1:]

def relax(m, e, w): # solving a system of equations by the over-relaxation method
    x = np.zeros(len(m))
    count = 0

    while norm(residual(m, x)) >= e:
        for i in range(len(m)):
            sum = 0
```

```

        for j in range(len(m)):
            sum = sum + m[i, j] * x[j]

        x[i] = x[i] - w / m[i, i] * sum + w * m[i, len(m)] / m[i, i]

    count += 1

    print('number of iterations:', count)

    return x

def det(m): # calculation of the determinant of a matrix
    new_m = np.copy(m)
    res = 1

    n = len(m)
    for i in range(n - 1):
        if len(np.nonzero(abs(new_m[i:, i]) > EPS)[0]) == 0:
            return 0

        nonzero = np.nonzero(abs(new_m[i:, i]) > EPS)[0][0]
        + i
        new_m[[i, nonzero]] = new_m[[nonzero, i]]

        if (i != nonzero):
            res = res * -1

        res = res * new_m[i][i]
        new_m[i] = new_m[i] / new_m[i][i]
        new_m[i + 1:, :] = new_m[i + 1:, :] - np.matmul(np.transpose(new_m[i + 1:,
                                                                           i])[:, np.newaxis], new_m[i, np.newaxis])

    res = res * new_m[n - 1][n - 1]
    return res

def norm(m): # finding the norm of a matrix
    return np.max(np.sum(abs(m[:, : len(m)]), axis = 1))

```

```

def main():
    print('Please choose how you want to enter the matrix')
    print('>>I want to enter the matrix manually (1)')
    print('>>I want to enter the matrix by the formula (2)')
    mode = int(input('mode (1 or 2) = '))

    n = int(input('n = '))
    m = []

    if mode == 1:
        for i in range(n):
            m.append([float(j) for j in input('current row: ').split()])
    else:
        m = form_matr(n)
    print("\nInput matrix:") # output of the entered matrix
    for row in m:
        print(' '.join(list(map(str, row))))
    m = np.array(m, dtype = float)
    det_m = det(m)

    e = float(input('\nEps = '))
    print("\nSolution of a system of equations by the over-relaxation method")

    if det_m != 0:
        if (not is_pos(m)) or (not is_herm(m)):
            m_trans = m[:, : -1].transpose()
            new_left = m_trans.dot(m[:, : -1])
            new_right = m_trans.dot(m[:, -1:])
            m = np.concatenate((new_left, new_right), axis = 1)

            for w in np.arange(0.1, 2.0, 0.1):
                print('with w =', w, relax(m, e, w), end = '\n\n')

        else:
            print('does not exist\n')
            print("determinant of a matrix:")
            print(det_m, end = '\n\n')

if __name__ == '__main__':
    main()

```

## Тесты

### Тест 1

$$\begin{cases} 2x_1 - x_2 - 6x_3 + 3x_4 = -1, \\ 7x_1 - 4x_2 + 2x_3 - 15x_4 = -32, \\ x_1 - 2x_2 - 4x_3 + 9x_4 = 5, \\ x_1 - x_2 + 2x_3 - 6x_4 = -8. \end{cases}$$

|                      | Результат программы        | Реальное значение  |
|----------------------|----------------------------|--------------------|
| Решение СЛАУ         | (-3, 0, -0.500.., 0.667..) | (-3, 0, -1/2, 2/3) |
| Определитель матрицы | 2.9999999999999993         | 3                  |

Положим  $\varepsilon = 0.000001$ . Наилучшая сходимость при  $\omega = 1.2$  и  $1.3$

| $\omega$ | Число итераций | $\omega$ | Число итераций | $\omega$ | Число итераций | $\omega$ | Число итераций |
|----------|----------------|----------|----------------|----------|----------------|----------|----------------|
| 0.1      | 16318          | 0.6      | 1674           | 1.1      | 748            | 1.6      | 226            |
| 0.2      | 7587           | 0.7      | 1202           | 1.2      | 628            | 1.7      | 154            |
| 0.3      | 4663           | 0.8      | 1174           | 1.3      | 520            | 1.8      | 231            |
| 0.4      | 3203           | 0.9      | 1027           | 1.4      | 419            | 1.9      | 475            |
| 0.5      | 2324           | 1.0      | 881            | 1.5      | 324            |          |                |

### Тест 2

$$\begin{cases} x_1 - 2x_2 + x_3 + x_4 = 0, \\ 2x_1 + x_2 - x_3 - x_4 = 0, \\ 3x_1 - x_2 - 2x_3 + x_4 = 0, \\ 5x_1 + 2x_2 - x_3 + 9x_4 = -10. \end{cases}$$

|                      | Результат программы                         | Реальное значение                        |
|----------------------|---------------------------------------------|------------------------------------------|
| Решение СЛАУ         | (-0.254..., -0.763..., -0.424..., -0.847..) | (-0.254..., -9090.., -0.588..., -1.11..) |
| Определитель матрицы | -117.889..                                  | -118                                     |

Положим  $\varepsilon = 0.0000001$ . Наилучшая сходимость при  $\omega = 0.7$

| $\omega$ | Число итераций | $\omega$ | Число итераций | $\omega$ | Число итераций | $\omega$ | Число итераций |
|----------|----------------|----------|----------------|----------|----------------|----------|----------------|
| 0.1      | 2890           | 0.6      | 373            | 1.1      | 132            | 1.6      | 65             |
| 0.2      | 1383           | 0.7      | 299            | 1.2      | 106            | 1.7      | 86             |
| 0.3      | 880            | 0.8      | 243            | 1.3      | 81             | 1.8      | 140            |
| 0.4      | 627            | 0.9      | 199            | 1.4      | 53             | 1.9      | 295            |
| 0.5      | 475            | 1.0      | 163            | 1.5      | 51             |          |                |

### Тест 3

$$\begin{cases} 8x_1 + 6x_2 + 5x_3 + 2x_4 = 21, \\ 3x_1 + 3x_2 + 2x_3 + x_4 = 10, \\ 4x_1 + 2x_2 + 3x_3 + x_4 = 8, \\ 7x_1 + 4x_2 + 5x_3 + 2x_4 = 18. \end{cases}$$

|                      | Результат программы   | Реальное значение     |
|----------------------|-----------------------|-----------------------|
| Решение СЛАУ         | (1.0, 2.0, -2.0, 0.0) | (1.0, 2.0, -2.0, 0.0) |
| Определитель матрицы | -242.0                | -242.0                |

Положим  $\varepsilon = 0.000001$ . Наилучшая сходимость при  $\omega = 1.8$

| $\omega$ | Число итераций | $\omega$ | Число итераций | $\omega$ | Число итераций | $\omega$ | Число итераций |
|----------|----------------|----------|----------------|----------|----------------|----------|----------------|
| 0.1      | 2851           | 0.6      | 256            | 1.1      | 213            | 1.6      | 394            |
| 0.2      | 1310           | 0.7      | 157            | 1.2      | 316            | 1.7      | 229            |
| 0.3      | 793            | 0.8      | 163            | 1.3      | 679            | 1.8      | 403            |
| 0.4      | 532            | 0.9      | 159            | 1.4      | 434            | 1.9      | 398            |
| 0.5      | 371            | 1.0      | 176            | 1.5      | 471            |          |                |

## Тест 4

Элементы матрицы  $A$  вычисляются по формулам:

$$A_{ij} = \begin{cases} q_M^{i+j} + 0.1 \cdot (j-i), & i \neq j, \\ (q_M - 1)^{i+j}, & i = j, \end{cases}$$

где  $q_M = 1.001 - 2 \cdot M \cdot 10^{-3}$ ,  $i, j = 1, \dots, n$ .

$$f_i = x \cdot \exp\left(\frac{x}{i}\right) \cdot \cos\left(\frac{x}{i}\right)$$

$M = 6, N = 100$

Результаты программы:

Определитель - 2.239191618567845e-43

Наилучшая сходимость наблюдается при  $\omega$  между 0.8 и 0.9. Результаты программы близки к результатам работы библиотечных функций.

## Вывод

В ходе выполнения отчетной работы был рассмотрен итерационный метод верхней релаксации для решения СЛАУ. Как было показано, скорость сходимости зависит от итерационного параметра, поэтому подбор наилучшего значения является важной задачей. Также в качестве достоинства данного метода стоит отметить относительно простую алгоритмизацию и реализацию в программе.